Process & Decision Documentation

Name: Giang Tran
Role(s): Sole Coder
Primary responsibility for this work: All tasks for Week 3 Side Quest 2 - Platform Game Modification

Goal of Work Session

Modify the existing AABB collision platform game to express "Mischievous Panic" - the anxious excitement of getting away with something you shouldn't be doing.

Tools, Resources, or Inputs Used

- p5.js Web Editor / Local Environment
- Base Code: Week 2 Example 3 by Karen Cochrane and David Han
- Lecture Notes (Week 2 & 3)
- p5.js Reference Documentation
- GitHub (version control)

GenAI Documentation

GenAI was NOT used for this assignment. All code modifications, game mechanics, and emotional design were developed through manual coding, testing, and iteration using trial-and-error debugging and p5.js documentation.

Summary of Process

Development Steps:

1. **Added Collectible Items** - Created `items` array with coins, gems, and stars scattered on platforms
2. **Implemented Stealing Mechanic** - Used distance calculation to detect when blob touches items
3. **Built Guilt System** - `blob3.guilt = constrain(blob3.stolenItems / 4, 0, 1)` scales anxiety with thefts

4. **Added Nervous Behaviors** - Jittery movement, idle nervous hops, sweat drops
5. **Created Item Physics** - Items bump away when blob touches them, fall with gravity
6. **Visual Polish** - Different item types rotate/glow, sweat wobbles, proximity detection

Key Bugs & Solutions:

Bug 1: Items re-stolen every frame

- Fixed by adding `if (item.stolen) continue;` check at loop start

```
      for (let item of items) {
        if (item.stolen) continue;

```

Bug 2: Items fell through platforms

- Copied blob collision logic and adapted for item bounding boxes

```
      // Platform collision for items
      for (const p of platforms) {
        let itemBox = {
          x: item.x - item.size / 2,
          y: item.y - item.size / 2,
          w: item.size,
          h: item.size,
        };

        if (overlap(itemBox, p)) {
          if (item.vy > 0) {
            item.y = p.y - item.size / 2;
            item.vy = 0;
          }
        }
      }

```

Bug 3: Jitter made blob teleport

- **v1:** `blob3.x += random(...)` → teleporting chaos
- **v2:** `blob3.vx += random(-0.1, 0.1)` → too subtle

- **Final:** `blob3.vx += random(-0.3, 0.3)` → just right

```
    // Add random jitter to movement when guilty
    let jitter = random(-blob3.guilt * 0.3, blob3.guilt * 0.3);
    blob3.vx += blob3.accel * move + jitter;
```

Bug 4: Rotation spun entire canvas

- Forgot `push()/pop()` around item rotation code, took 10 min to figure out

```

  // --- Draw items ---
  for (let item of items) {
    if (item.stolen) continue;

    // Items glow/pulse when blob is near
    let nearGlow =
      blob3.nearItem && dist(blob3.x, blob3.y, item.x, item.y) < 60;

    push();
    translate(item.x, item.y);

    if (item.type === "coin") {
      fill(255, 220, 0, nearGlow ? 255 : 200);
      ellipse(0, 0, item.size, item.size);
      fill(255, 240, 100);
      ellipse(0, 0, item.size * 0.5, item.size * 0.5);
    } else if (item.type === "gem") {
      fill(255, 100, 200, nearGlow ? 255 : 200);
      rotate(frameCount * 0.02);
      for (let i = 0; i < 6; i++) {
        rotate(PI / 3);
        triangle(0, -item.size / 2, item.size / 3, 0, -item.size / 3, 0);
      }
    } else if (item.type === "star") {
      fill(100, 255, 255, nearGlow ? 255 : 200);
      rotate(frameCount * 0.03);
      for (let i = 0; i < 5; i++) {
        rotate(TWO_PI / 5);
        triangle(0, -item.size / 2, item.size / 5, -item.size / 5, 0, 0);
      }
    }

    pop();
  }
```

Bug 5: Bump physics sent items flying

- Tested pushForce values: 5 = too strong, 0.5 = too weak, 2 = perfect

```
        if (d < 60) {
          blob3.nearItem = true;
        }

        // BUMP PHYSICS: Items get knocked around by blob
        if (d < blob3.r + item.size + 15) {
          let angle = atan2(item.y - blob3.y, item.x - blob3.x);
          let pushForce = 2;
          item.vx += cos(angle) * pushForce;
          item.vy += sin(angle) * pushForce;
        }
```

Decision Points & Trade-offs

Decision 1: Guilt scaling (divide by 4 vs 10)

- Tested 3, 4, 5, 6 - chose 4 so players hit max panic before collecting all 7 items

```
    // --- Update guilt level based on stolen items (0 = calm, 1 = max panic) ---
    blob3.guilt = constrain(blob3.stolenItems / 4, 0, 1);
```

Decision 2: Nervous hop frequency

- `random() < 0.1` = too frequent (epileptic)
- `random() < 0.01` = too rare (barely noticeable)
- `random() < 0.02` = occasional nervous twitch ✓

```
// --- Track idle time for spontaneous nervous behavior ---
if (abs(blob3.vx) < 0.5 && blob3.onGround) {
  blob3.idleTime++;
  // Spontaneous nervous hop when anxious and idle
  if (blob3.idleTime > 60 && blob3.guilt > 0.3 && random() < 0.02) {
    blob3.vy = -6;
    blob3.idleTime = 0;
  }
} else {
  blob3.idleTime = 0;
}

```

Decision 3: Visual emotion (color change vs external cues)

- Chose sweat/shake/jitter over color change because it's more readable as anxiety

```
if (blob3.guilt > 0.3) {
  let sweatX = blob3.x + 15 + sin(frameCount * 0.2) * 2;
  let sweatY = blob3.y - blob3.r - 10;
  fill(100, 200, 255, blob3.guilt * 200);
  ellipse(sweatX, sweatY, 5, 7);
}

```

```
// --- Nervous shake increases with guilt ---
blob3.nervousShake = sin(frameCount * 0.3) * blob3.guilt * 3;

```

```
let jitter = random(-blob3.guilt * 0.3, blob3.guilt * 0.3);
blob3.vx += blob3.accel * move + jitter;
```

Verification & Testing

Tested all scenarios:

- Items collide with platforms correctly
- Guilt scales from calm (0 items) to max panic (4+ items)

- Blob shows nervous behaviors when idle at high guilt
- Mid-air stealing works
- Items scatter realistically when bumped

Dead Ends & Limitations

Abandoned features:

1. Camera follow system - broke collision detection with translate() offset
2. Particle explosions when stealing - made code too cluttered
3. Sound effects - no time to source audio files

Potential improvements:

- Guilt decay over time
- Environmental reactions (shaking platforms)
- Multiple emotional states (panic vs calm paths)

Code Examples Showing Human Process

Bump Physics Evolution:

javascript

```
// First try: random velocity (wrong direction!)
item.vx += random(-5, 5);


// Final: calculated angle from blob to item
let angle = atan2(item.y - blob3.y, item.x - blob3.x);
item.vx += cos(angle) * pushForce;
```

Collision Detection Fix:

javascript

```
// Initial: hardcoded distance (doesn't scale)
if (dist(...) < 20) { }

// Final: accounts for different sizes
if (d < blob3.r + item.size) { }
```

Sweat Drop Wobble (Visual Polish):

javascript

```
// First version: static sweat drop (boring)
let sweatX = blob3.x + 15;
let sweatY = blob3.y - blob3.r - 10;

// Added wobble for nervous effect
let sweatX = blob3.x + 15 + sin(frameCount * 0.2) * 2;
// Tried 0.5 at first (moved 1 pixel - barely visible)
// Then tried 5 (swung wildly - looked wrong)
// Settled on 2 (subtle trembling effect)
```

Item Stealing Bug Fix:

javascript

```
// BROKEN: Items got re-stolen every frame!
for (let item of items) {
  let d = dist(blob3.x, blob3.y, item.x, item.y);
  if (d < blob3.r + item.size) {
    item.stolen = true;  // This runs 60 times per second!
    blob3.stolenItems++; // Count kept increasing!
  }
```

```
}

// FIXED: Check if already stolen first
for (let item of items) {
  if (item.stolen) continue; // Skip if already taken
  let d = dist(blob3.x, blob3.y, item.x, item.y);
  if (d < blob3.r + item.size) {
    item.stolen = true;
    blob3.stolenItems++;
  }
}
```